

REPORT ON SMART HOME LIGHTING SYSTEM ARCHITECTURE AND OPERATIONAL LOGIC

1. SYSTEM OVERVIEW & INTRODUCTION

1.1 Executive Summary

This report provides a comprehensive architectural and operational breakdown of the Smart Home Lighting System as detailed in **137049.png**. The system integrates modern microcontrollers, sensor arrays, and multi-channel relay modules to deliver an energy-efficient, automated, and user-responsive residential illumination network. By decoupling the control logic from direct manual switching, the architecture enables dynamic brightness adjustment, environmental sensing, and remote orchestration via internet-connected devices.

1.2 System Scope & Core Objectives

The primary objective of the engineered system is to maximize user comfort and minimize energy waste. As illustrated under the "KEY FEATURES" section of **137049.png**, the system fulfills six core requirements:

- **Remote Control via Mobile App:** Facilitates off-site monitoring and manual overriding of the lighting infrastructure.
- **Motion Detection Auto Lighting:** Uses spatial occupancy indicators to dynamically activate or deactivate localized illumination zones.
- **Scheduling & Timers:** Executes time-deterministic actions to simulate occupancy or adhere to routine behavior.
- **Ambient Light Sensing:** Samples environmental illuminance to prevent daytime over-lighting.
- **Dimming & Scene Management:** Adjusts voltage or duty cycles to provision customized ambient profiles.
- **Energy Efficient & Smart:** Merges all variables into an optimization matrix to reduce aggregate power draw.

2. HARDWARE ARCHITECTURE & CIRCUIT CONNECTIONS

2.1 Central Controller Module

The core computational engine of the architecture is the **ESP32 / MCU** block. According to the system architecture diagram in **137049.png**, this controller natively integrates four vital sub-components:

- **Wi-Fi Module:** Manages network layer communications to exchange telemetry data with cloud brokers or mobile applications.

- **I/O Control:** General Purpose Input/Output (GPIO) pins mapped to intercept sensor signals and trigger driver logic.
- **Logic Processing:** The central processing unit executing the algorithmic firmware loop.
- **Memory:** Non-volatile storage containing user configurations, static schedules, and boundary thresholds.

2.2 Power Delivery Network

The system utilizes a dual-stage power architecture to isolate high-voltage mains from low-voltage digital electronics:

1. **AC Source:** A standardized input rating of **AC 100–240V, 50/60Hz** feeds the primary lines.
2. **DC Step-Down:** The power supply converts the mains into a regulated **5V DC** rail. As shown in **137049.png**, this continuous power line feeds directly into the main ESP32 controller block.

2.3 Peripheral Interfaces (Inputs & Outputs)

The circuit topology separates data acquisition from load management through predefined signal paths:

Module Class	Component	Connection Type	Description
Inputs	Mobile App (User)	Wireless (Wi-Fi)	Direct API/MQTT commands sent to the controller.
Inputs	Wall Switch	Digital Signal	Physical toggle override connected to a GPIO pin.
Inputs	Motion Sensor (PIR)	Digital/Interrupt	Signals spatial occupancy transitions.
Inputs	Light Sensor (Ambient)	Analog/I2C	Provides real-time lux data to compute required dimming.
Inputs	Schedule/Timer	Internal/NTP	Synchronized time interrupts for automated events.

Module Class	Component	Connection Type	Description
Outputs	Relay / Driver (Ch 1)	Digital Output	Drives independent circuit for Light 1 .
Outputs	Relay / Driver (Ch 2)	Digital Output	Drives independent circuit for Light 2 .
Outputs	Relay / Driver (Ch n)	Digital Output	Scaleable driver channel for the n-th Light fixture.

3 SOFTWARE FIRMWARE LOGIC & OPERATIONAL FLOW

3.1 Initialization Sequence

Upon cold booting or receiving a hardware reset command, the firmware executes a sequential setup routing as defined in the **137049.png** flowchart:

1. **Check Power:** Confirms voltage levels across the digital rails are stable.
2. **Connect to Wi-Fi:** Initializes the network stack and attempts handshake protocols with the designated access point.
3. **Initialize Sensors:** Calibrates the Passive Infrared (PIR) sensor and reads initial values from the ambient light photo-resistor.
4. **Load Settings:** Pulls user parameters, scenes, and scheduling schemas from local flash memory.

3.2 Main Algorithmic Loop (Decision Tree)

Once initialization completes, the controller steps into a continuous, non-blocking **REPEAT (Loop)** sequence. The decision matrix evaluates conditions in a specific hierarchical order:

- **Priority 1: User Command Received?**
 - *If Yes:* The system breaks into the **Process Command** subroutine. It handles manual overrides: ON/OFF states, Dim / Brighten adjustments, Scene Selection, or Schedule Setting updates.
- **Priority 2: Motion Detected?**
 - *If Yes (and no manual override suppresses it):* The system navigates to **Turn On / Adjust Lighting**. It reads the ambient light sensor data to *Determine Brightness* levels and immediately *Applies Light Level* variations to the respective output channel.

- **Priority 3: Schedule Trigger?**
 - *If Yes:* The system executes the scheduled action block (**Execute Scheduled Action**), altering state parameters such as forcing hard ON/OFF cycles or updating the active Dim/Scene setting.
- **State Maintenance:** If all three conditional checkpoints return negative (**No**), the controller executes the **Maintain Current State** function, preserving existing duty cycles across all relay channels before recycling back to the start of the loop.

4 ARCHITECTURAL LEGEND & SYSTEM SCALABILITY

4.1 Interface Topology Legend

As defined by the schematic conventions in **137049.png**, the system categorizes physical and logical layers using distinct graphical indicators:

- **Green Blocks:** Represent Input / Source hardware modules that gather situational data or receive human interaction commands.
- **Purple Blocks:** Represent the Controller hardware core tasked with real-time compute operations.
- **Blue Blocks:** Represent Process / Driver blocks executing power-switching actions.
- **Yellow Blocks:** Map Power distribution nodes.
- **Dashed Arrow Lines (\$-->\$):** Map the unidirectional or bidirectional **Signal / Flow** paths transporting data packets or logic levels.
- **Solid Solid Lines (—):** Represent direct physical **Power Connections** stemming from the 5V DC supply rail.

4.2 Scalability and Extensibility Matrix

The system design emphasizes modular scale. Because the controller isolates output stages using independent multi-channel abstractions (**Ch 1, Ch 2, Ch 3 ... Ch n**), the infrastructure can expand seamlessly from a single-room deployment to an entire enterprise facility.

Adding an output node requires mapping a new driver channel to an open GPIO pin on the MCU, while the firmware's polling loop autonomously applies the identical decision ensuring robust, uniform behavior across all connected fixtures.